Porting the Autoware Safety Island onto nano-ros: a Reference-Consumer Migration Study

NEWSLab, National Taiwan University

Technical report for the Autoware Foundation

Abstract. The Autoware Safety Island (ASI) runs Autoware's trajectory follower — MPC lateral and PID longitudinal control — as a standalone application on a safety-class Arm processor, exchanging commands with a full Autoware stack over DDS. This report describes the current state of ASI's migration from a hand-glued CycloneDDS port onto *nano-ros*, a lightweight ROS 2 client library for embedded RTOSes, contrasting the original firmware with the nano-ros-enabled one. The migration was bidirectional: ASI acts as nano-ros's *reference consumer*, and nineteen consumer-surfaced defects were fixed upstream or filed as tracked issues along the way — the effort of making nano-ros fit a real safety application is part of the result. We summarize what nano-ros enables (an unmodified vendored controller behind an rclcpp-faithful API, declarative bringup with a generated entry, wire-level ROS 2 interoperability by construction, bounded-memory typed messaging), the concrete before/after differences in the firmware and repository, the Arm FVP integration, the validation evidence (six build modes, a five-phase emulator runtime suite, a closed-loop demonstration against unmodified ROS 2 Humble Autoware), and the gaps that remain on both sides.

1 Introduction

Safety architectures for autonomous driving separate a *safety island* — a minimal, certifiable control path on a lockstep-capable processor — from the performance-oriented compute domain running the full autonomy stack. ASI realizes this pattern for Autoware: the trajectory follower (MPC lateral and PID longitudinal controllers, vendored unmodified from Autoware) runs on an Arm Cortex-R82 target under Zephyr RTOS, consumes the planned trajectory and vehicle state over DDS, and returns `autoware_control_msgs/Control` commands (optionally mirrored onto CAN).

The original port owned its middleware: a vendored CycloneDDS tree, host IDL tooling, and a bespoke node/executor shim — exactly the code a safety argument least wants to own. nano-ros replaces that glue with a maintained, wire-compatible middleware layer. The migration was deliberately bidirectional: rather than adapting ASI around middleware limitations, we fixed nano-ros where it did not yet fit a real safety application. Every integration failure ("wall") was reproduced minimally and either landed as an upstream fix or was filed as a tracked nano-ros issue; downstream workarounds exist only as stop-gaps designed to retire themselves.

2 System overview

Figure 1 shows the validated closed loop. An unmodified Autoware container (ROS 2 Humble, `rmw_cyclonedds`) runs the planning simulator on DDS domain 1; a DDS bridge relays the five controller inputs to domain 2, pinned to a host TAP interface; the island on the Arm FVP consumes them and publishes control commands back through the same path. There is no type adaptation anywhere in the path.

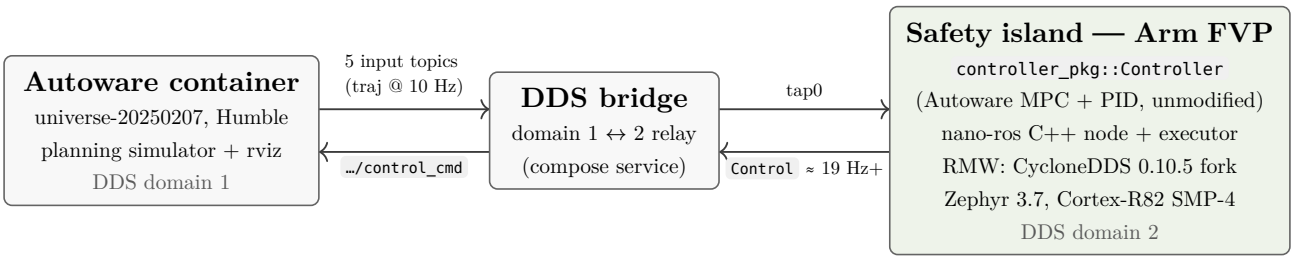

Figure 1: The closed-loop demo (`scripts/run-tap-demo.sh`). The island's NIC model attaches to the host TAP; both DDS sides run stock CycloneDDS wire format.

3 What nano-ros enables

The advantages the migration bought, in decreasing order of importance to the safety argument:

- **The vendored controller stays unmodified.** nano-ros's C++ node surface is rclcpp-faithful — parameter declaration, member-function-pointer subscriptions and timers, value-typed publishers — so Autoware's MPC and PID components compile against it as they are. The port's entire change surface lives in adapter seams, which keeps the provenance argument for the control algorithms intact.
- **Declarative bringup, generated entry.** The node topology, remappings and 74 launch parameters are authored as data (`system.toml` + launch XML), resolved into a reproducible system model, and baked into the bootable image by one CMake call (Figure 2). No hand-written boot code exists; what the image runs is, by construction, what the bringup declares — the property a safety review actually wants to check.
- **ROS 2 interoperability by construction, not by bridge logic.** nano-ros pins its CycloneDDS to the release ROS Humble ships and generates wire-compatible types from the same `.msg` sources, so the island talks to an unmodified Autoware stack with no translation layer (verified in both directions in the demo).
- **Bounded memory on the data path.** Generated message types carry fixed-capacity sequences and strings (e.g. a 250-point trajectory bound), and every controller input is bound to KEEP_LAST depth 1 — no unbounded queues or heap growth on the receive path; sizing is set at build time and verified under a real trajectory stream.
- **A maintained middleware instead of a private fork.** CycloneDDS-on-Zephyr fixes (multicast join, mutex-pool exhaustion, thread bring-up) now live upstream where they are tested continuously, instead of in ASI patches. The same API also spans FreeRTOS/NuttX/ThreadX and two further RMW backends, which keeps ASI's future porting surface small.
- **Reproducible provisioning.** Toolchains, the RMW source, patch sets and host tools are declared as data and applied by idempotent commands; ASI's host bootstrap is thin and robust against upstream restructuring.

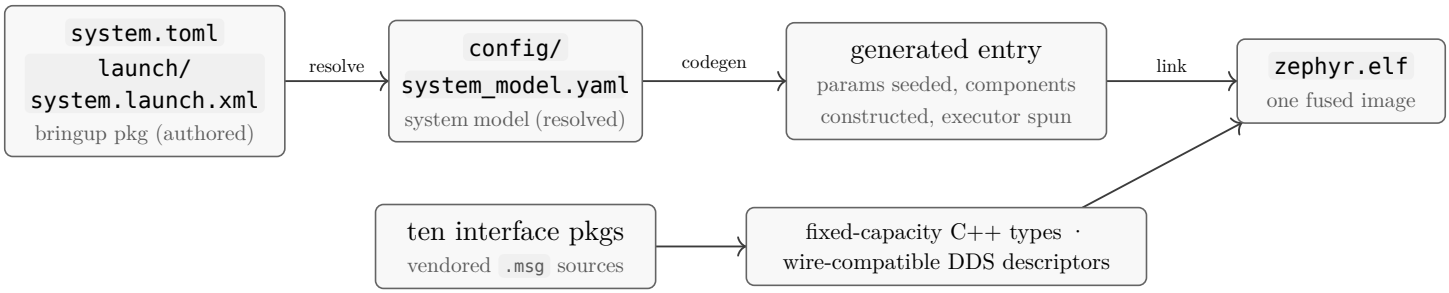

Figure 2: Declarative bringup to bootable image: topology and interfaces are data; everything executable is generated.

4 From the original ASI to the nano-ros ASI

Table 1 contrasts the two firmwares seam by seam. The common thread: in the original, every seam was *owned code*; in the nano-ros ASI it is either *declared data* or *generated*, and the only hand-written C++ left outside the vendored controller is thin adapters.

Seam	**Original ASI**	**nano-ros ASI**
Middleware	vendored CycloneDDS tree, patched in-repo; hand-built host `idlc`	nano-ros submodule in lockstep with the west pin; CycloneDDS fork and host tools provisioned by the nano-ros CLI
Boot	imperative `main.cpp`: network wait, SNTP, node construction, banner prints	generated entry baked from the resolved system model; a weak network hook and the component constructor carry the app-specific parts
Node API	bespoke `Node` class: raw descriptor-based pub/sub, own pthread poll loop, ad-hoc parameter map	`nros::ComponentNode`, rclcpp-faithful; the controller registers as a component under the package/class identity rule; the polling shim survives only for test images
Launch & params	topics and 150+ parameters hardcoded or seeded in C++	authored in `system.toml` + launch XML, resolved to a committed system model; 74 launch parameters seeded by the generated entry
Messages	idlc-generated C structs; raw `_buffer/_length` sequences, manual memory management in tests and adapters	generated C++ value types with fixed-capacity sequences/strings; one umbrella header maps ASI's aliases; descriptors wire-compatible with `rmw_cyclonedds`
QoS	deep default histories (up to 500) — stale buffered trajectories	KEEP_LAST depth 1 on every controller input, audited under a real >1400-byte trajectory stream
Host workflow	hand-maintained scripts per concern; devcontainer-oriented	thin no-sudo bootstrap (`scripts/bootstrap-asi.sh`), one-command demo (`scripts/run-tap-demo.sh`), CI runtime suite that also runs on a developer host
Portability	one path: Zephyr + vendored CycloneDDS	board-bundle import (one CMake call per board); the same app API spans four RTOSes and three RMW backends upstream

Table 1: Firmware and repository diff, original vs. nano-ros-enabled ASI. Vendored Autoware component logic: unchanged in both.

5 Migration status

Current state. The Zephyr targets are fully migrated: the FVP reference platform builds, boots and closes the loop on nano-ros with no legacy middleware in the image. The FreeRTOS targets still run the original CycloneDDS path unchanged (their migration is scoped, not started). All six firmware build modes, the five-phase FVP runtime suite, and the closed-loop demo pass on the current pin; control-command output rates match the original port's baseline ($\approx$ 19 Hz and above).

What the port surfaced. Making nano-ros fit ASI produced nineteen precisely-reproduced defects. Most were fixed on nano-ros main and are now covered by its own CI — among them Zephyr multicast join in the CycloneDDS fork, mutex-pool exhaustion against a 40-participant Autoware graph, parameter-pool and subscription-buffer capacity for a controller of this size, and per-package message FFI generation. Two remain open as tracked nano-ros issues with self-retiring ASI workarounds (Table 2 #1, #4); a further group were latent ASI validation gaps that full re-validation exposed (#6, #9–11) — defects of the original port, not of nano-ros. None of the previously fixed defects has regressed on the current pin.

#	Symptom	Root cause	Disposition
1	Board provisioning command rejects the board	CLI resolves a retired directory layout; the bundle-aware resolver exists but two verbs never got it	nano-ros issue #0729; steps inlined downstream
2	Configure fails: host IDL compiler not found	Host tooling moved to the CLI-managed SDK store	One provisioning command; bootstrap updated
3	Board-facts/runtime-config rung silently absent	Resolver cannot locate the nano-ros checkout from an out-of-tree consumer	Env pointer exported; residual is #0729′s class
4	C++ runtime fails to compile for the embedded target	Feature composition omits `alloc` where the error path needs it; simulator coverage masks it	nano-ros issue #0730; self-retiring downstream patch
5	Test programs: missing headers, unknown type names	Legacy flat-name compatibility layer retired; typed API is now the only surface	Tests migrated onto ASI's message umbrella
6	CAN filter flag undeclared	Zephyr 3.7 removed it; the CAN test predated the 3.7 move	Standard-ID filter is `flags = 0`
7	Entry panic policy changed	Upstream default moved from halt to the platform's fatal path	Accepted — loud fatal suits a safety island
8	Sizing knobs went live	Previously inert config knobs are now forwarded	Audited; build-time sizing still authoritative
9	Boot liveness markers never printed	Markers lived in the retired hand-written boot code	Component constructor now owns them
10	Test images fail at node creation	Entry-less images never initialized the runtime	Shim performs one-time init
11	Test hangs on node stop	Thread cancellation never lands on Zephyr's POSIX layer	Cooperative stop flag

Table 2: Defects surfaced while bringing ASI onto the current nano-ros pin (the earlier-resolved defects live in nano-ros's tracker). Full detail in the repository's phase-3 roadmap document.

6 FVP integration

The Arm FVP_BaseR_AEMv8R (Cortex-R82, SMP-4) is the reference platform. nano-ros ships the board as a bundle — board id, toolchain, default RMW, Kconfig/DTS fragments and runner hint behind one CMake call — so ASI carries only its own deltas (application sizing, TAP profile). Operational facts that matter for reproduction:

- The model's NIC is off by default; the build enables it, and two network profiles exist: FVP user-mode networking with DHCP (used by CI) and the TAP profile pinning DDS domain 2 to `tap0` (used by the demo).
- `west build --target run` launches the model through Zephyr's standard runner — no hand-rolled launch scripts. One model flag matters: `cache_state_modelled=0`; the default makes busy code $\approx 1000\times$ slower and presents as a hang at network bring-up.
- CI runs a five-phase runtime suite against a sha-pinned FVP download: controller boot to its liveness markers, the on-target unit suite, DDS loopback, CAN output, and the TAP image build. The same script runs unchanged on a developer host.
- The full image links to 9.7 MB of the model's 128 MB RAM, with parameter, executor and subscription-buffer capacity set at build time.

7 Running the demo

The closed loop of Figure 1 reproduces on any x86-64 Linux host with Docker and the (license-gated, free) Arm FVP:

1. **Provision the host** (once): `scripts/bootstrap-asi.sh` installs west, the Zephyr SDK, the nano-ros CLI and board provisioning without ever invoking sudo, then `source ./activate-asi.sh`. Place the FVP under `tools/fvp/` or set `ARMFVP_BIN_PATH`.
2. **Create the TAP interface** (once, the only root step): `sudo scripts/setup-tap.sh` (idempotent; `--delete` to remove).
3. **Run:** `scripts/run-tap-demo.sh`. The script builds the TAP image (incrementally), starts the compose stack (Autoware planning simulator, DDS bridge, visualizer), boots the island on the FVP and waits for its liveness markers, seeds the ego pose and a goal headless (the sample-map coordinates from the project runbook), and verifies `/control/trajectory_follower/control_cmd` streams on the Autoware side — printing `CLOSED LOOP OK` with the measured rate.
4. **Observe / stop:** the island keeps running for interactive use (`log/tap-demo-fvp.log`; rviz via the visualizer container); `scripts/run-tap-demo.sh --down` stops the FVP and the stack.

Manual seeding and probing (e.g. from rviz, or `ros2 topic pub` / `hz` inside the Autoware container) are documented in `demo/README.md` and the phase-3 runbook; `--no-seed` brings the stack up without seeding.

8 Validation

Build: six firmware modes from one branch (full controller, unit test, DDS loopback, CAN output, DDS publisher/subscriber). **Runtime:** the five-phase FVP suite passes end to end. **Closed loop:** with the stack of Figure 1 brought up headless, the planner streams the trajectory at 10 Hz and the island returns control commands at 19 Hz and above — matching the original port's baseline; from the spawn position the MPC correctly commands an emergency stop on excessive tracking error, i.e. the controller logic is demonstrably live. Interoperability was spot-checked at field level from the Humble side. `scripts/run-tap-demo.sh` reproduces the run, including the rate check, in one command.

9 Remaining gaps

On the nano-ros side:

- **Board provisioning verbs miss bundle boards** (#0729) — the sanctioned downstream provisioning path fails for every in-tree Zephyr board until the bundle-aware resolver reaches it; ASI inlines the steps meanwhile.
- **Embedded C++ CycloneDDS composition** (#0730) — the missing `alloc` capability, a coverage gap of exactly ASI's coordinate (embedded × C++ × CycloneDDS); ASI carries a self-retiring patch.
- **No S32Z270 board bundle yet** — hardware parity for ASI's second target waits on it.
- **Entry-level policy expression** — the panic policy (and similar image-level choices) cannot yet be expressed through the verb ASI uses.
- **Runtime-config rung for out-of-tree consumers** — board-facts resolution still assumes in-tree layouts.

On the ASI side:

- **FreeRTOS targets still run the legacy CycloneDDS path** — migrating them onto nano-ros's platform layer retires the vendored middleware entirely.
- **Services and actions are unexercised** — the island uses pub/sub and timers only; interop claims extend only that far.

- **Long-duration soak pending** — the real-run checkpoint scenario has not yet been re-run on the modern stack.
- **The demo stops at the control output** — the simulator's ego is not wired to consume the island's commands; closing that last link is a demo-wiring change, not a firmware one.

10 Conclusion

The migration replaced ASI's bespoke middleware with nano-ros and paid for it in the right currency: nineteen precisely-reproduced defects, most fixed upstream where they are now continuously tested, the rest tracked. In return the safety island gained an unmodified vendored controller behind a faithful API, a declaratively-generated image, wire-level ROS 2 interoperability with no translation layer, bounded memory on the data path, and a validation story (build, emulator runtime, closed loop) that one script reproduces. The reference-consumer arrangement — a real application continuously stress-testing a young middleware, with an explicit contract that fixes flow upstream — is, we believe, the transferable result.

Sources: `docs/roadmap/phase-3-modern-nano-ros-migration.md` (wall ledger, runbook), `docs/roadmap/phase-1*.md`, `docs/design/workspace_mode.rst`, nano-ros issues #0729/#0730. Repository: `github.com/newslabntu/autoware-safety-island`, branch `nano-ros`.